Comparing Dijkstra, A*, and JPS4 on 4-Connected Grid Maps

Algorithm Design & Analysis — Final Project Report

Jazib Waqas Salman Adnan Hunain Abbas
jw08048 sa07885 sh08466

Spring 2026

Abstract

We compare three shortest-path algorithms on uniform-cost 4-connected grid maps: Dijkstra's algorithm, A*, and JPS4, a recent adaptation of Jump Point Search for 4-directional movement. All three are guaranteed to return optimal paths. The question is how efficiently they do it. We ran 30 trials per density level across a sweep from 10% to 40% obstacle density on both 50×50 and 100×100 grids (420 problem instances per grid size). JPS4 consistently expands fewer nodes than both Dijkstra and A*, reducing expansions by 60–75% relative to Dijkstra across most density levels. The wall-clock gap is smaller due to JPS4's per-step overhead, and at very low obstacle density the advantage over A* shrinks. All algorithms are implemented in Python, verified against each other for path-length correctness on hundreds of random instances, and wrapped in an interactive Snake demo that lets a viewer watch the three algorithms race on the same board in real time.

1 Introduction

Shortest-path search on grid maps is one of the most common operations in game AI, robotics, and navigation systems. The choice of algorithm matters a lot in practice: a tile-based game calling a pathfinder every frame cannot afford an algorithm that explores most of the map to reach a nearby goal.

This project targets a specific, well-defined setting: a 4-connected grid where every step costs 1. That constraint rules out 8-directional movement, weighted edges, and any-angle paths. Within those boundaries, we compare three algorithms that all find optimal paths:

- **Dijkstra** – the uninformed baseline. No heuristic, expands outward in distance order.
- **A*** – adds Manhattan distance as a heuristic to guide search toward the goal.
- **JPS4** – applies symmetry pruning from Baum (2025) to skip intermediate nodes on straight corridors.

The core research question is: does JPS4's pruning give a meaningful practical advantage over A*, and under what conditions?

We built our own implementations of all three algorithms, wrote a test suite that checks path-length agreement across algorithms on hundreds of random instances, and ran a controlled benchmark across two grid sizes and seven density levels. The same code drives an interactive Snake demo where the algorithms can be watched side-by-side.

2 Background

2.1 Dijkstra's Algorithm

Dijkstra (1959) finds shortest paths in graphs with non-negative edge costs by expanding nodes in order of their distance from the source [1]. On a uniform-cost grid, this is equivalent to a wavefront that expands

outward in concentric rings. It is complete and optimal but has no goal-directed bias, so it tends to explore a large fraction of the reachable region before terminating.

2.2 A* Search

Hart, Nilsson, and Raphael (1968) augment Dijkstra with a heuristic estimate $h(n)$ of the remaining distance to the goal [2]. The node priority becomes $f(n) = g(n) + h(n)$, where $g(n)$ is the true cost from the start. With an admissible heuristic (one that never overestimates), A* is still optimal and typically expands far fewer nodes than Dijkstra by focusing the search toward the goal.

On a 4-connected grid with unit costs, Manhattan distance is the standard admissible heuristic:

$$h(n) = |x_n - x_{\text{goal}}| + |y_n - y_{\text{goal}}|$$

2.3 Jump Point Search and JPS4

Jump Point Search (JPS), introduced by Harabor and Grastien (2011) for 8-connected grids, addresses a different source of inefficiency [3]. Even with a good heuristic, A* still expands many *symmetric* paths: multiple routes that have the same cost but differ only in the order of moves along a straight corridor. JPS prunes these by identifying *jump points*, the only nodes at which a symmetric path can be distinguished from another.

Baum (2025) adapted this idea to 4-connected grids, producing JPS4 [4]. The key differences from classical JPS are a horizontal-first canonical ordering and revised pruning rules that account for the absence of diagonal movement. Horizontal moves always return all natural neighbors; vertical moves add forced neighbors only when an obstacle beside the parent would otherwise hide a shorter path around it. Jump points are then placed wherever an obstacle creates such a forced neighbor along the scan direction.

The result is that JPS4 can traverse long straight corridors in a single "jump" rather than expanding every cell along them, at the cost of more work per step.

3 Implementation

3.1 Core Algorithms

All three algorithms are implemented in `pathing_grid.py` around a shared A*-style priority queue loop (in `helper.py`). This design lets the three modes share the same correctness infrastructure and differ only in how they generate successors:

- **Dijkstra** uses the same 4-neighbor successor function as A* but passes a zero heuristic, so nodes are popped in pure distance order.
- **A*** uses Manhattan distance as the heuristic and expands all 4 passable neighbors at each step.
- **JPS4** replaces the neighbor expansion with a two-phase operation: (1) prune neighbors based on the incoming direction and adjacent obstacles, (2) jump along each pruned neighbor's direction until a jump point, obstacle, or the goal is found.

The JPS4 successor function returns jump points directly, potentially skipping over large swaths of the grid in a single step. The path reconstruction then fills in the intermediate cells.

3.2 Grid and Connectivity

The grid is a 2D Boolean array where `True` means blocked. The `PathingGrid` class maintains a Union–Find structure over free cells so that reachability queries (start/goal in the same connected component) run in near-constant time without a full BFS.

3.3 Snake Demo

The same `PathingGrid` drives an interactive Snake game built in Tkinter. The game runs three consecutive rounds on the same frozen board, one per algorithm, and displays each algorithm's search time, expansion count, and path length in a HUD strip below the grid. This lets a viewer see the behavioral difference between algorithms at the same density without confounding variables.

When the current food cell becomes temporarily unreachable (blocked by the snake's own body), the game falls back to a tail-following recovery strategy and relocates food to a reachable cell, keeping the demo stable without masking the pathfinder comparison.

3.4 Test Suite

The test suite in `tests/` covers four properties:

1. **Path length agreement:** Dijkstra, A*, and JPS4 return the same path length on every solvable instance sampled.
2. **Expansion ordering:** Dijkstra never expands fewer nodes than A* on the same instance (sanity check on the heuristic benefit).
3. **Reachable food selection:** the snake's food placement always picks a cell reachable from the snake's current head.
4. **Recovery correctness:** the previously reproducible unreachable-food loop no longer occurs.

All tests pass on the current codebase.

4 Experimental Setup

4.1 Grid Generation

Grids are synthetic: for each trial, every cell is independently set as blocked with probability ρ (the obstacle density). We use two grid sizes: 50×50 and 100×100.

Start and goal cells are placed to maximize path length. We first pick a random free cell, BFS to find the farthest reachable cell from it (call it A), then BFS again from A to find the farthest reachable cell from A (call it B). The trial uses A as start and B as goal. This "double BFS" trick avoids trivially short queries and produces instances that stress the algorithms more uniformly.

Each trial is run with all three algorithms on the same board. This controls for instance difficulty and lets us compare algorithms on identical queries.

4.2 Parameters

- **Grid sizes:** 50×50 and 100×100.
- **Density levels:** 0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40.
- **Trials per condition:** 30.
- **Seed:** fixed per density level (seed $= 99 + 7777 \times i + W$), ensuring reproducibility.
- **Metric – expansions:** the number of nodes popped from the priority queue (heap pops). This is the primary algorithmic efficiency metric, hardware-independent.
- **Metric – time:** Python `time.perf_counter` wall-clock time in milliseconds. These numbers are hardware-specific and noisier than expansion counts.

Instances where any algorithm fails to find a path are excluded from that trial (none occurred in this study; the Union–Find pre-filter ensures we only run solvable instances).

5 Results

5.1 Node Expansions

Table 1 and Table 2 report mean expansion counts across densities for both grid sizes. Figure 1 and Figure 2 show the full sweep visually.

Table 1: Mean node expansions on the 50×50 grid (30 trials per density).

Algorithm	$\rho = 0.10$	$\rho = 0.15$	$\rho = 0.20$	$\rho = 0.25$	$\rho = 0.30$	$\rho = 0.35$	$\rho = 0.40$
Dijkstra	2247.5	2121.6	1997.2	1858.6	1642.7	1241.7	624.3
A*	1949.7	1628.5	1352.5	1118.3	898.2	823.5	463.5
JPS4	570.5	639.2	627.5	590.7	512.6	497.0	281.7

Table 2: Mean node expansions on the 100×100 grid (30 trials per density).

Algorithm	$\rho = 0.10$	$\rho = 0.15$	$\rho = 0.20$	$\rho = 0.25$	$\rho = 0.30$	$\rho = 0.35$	$\rho = 0.40$
Dijkstra	8987.8	8490.4	7966.0	7434.1	6842.0	5605.7	2314.8
A*	7903.1	6439.7	5334.5	3839.3	3881.2	3493.8	1870.8
JPS4	2379.0	2500.1	2491.0	2031.6	2286.3	2103.4	1137.1

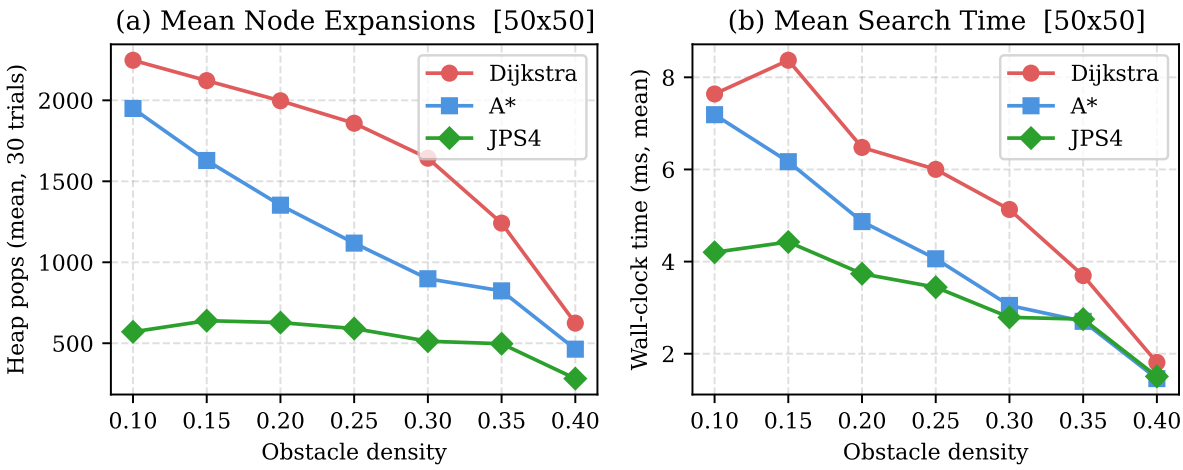

Figure 1: Mean expansions (left) and mean wall-clock time (right) on the 50×50 grid across seven density levels. Each point is the mean over 30 trials.

JPS4 consistently expands the fewest nodes at every density level on both grid sizes. On the 100×100 grid at $\rho = 0.10$, JPS4 expands 2,379 nodes versus 8,988 for Dijkstra and 7,903 for A*, a reduction of 73% relative to Dijkstra and 70% relative to A*. This large gap at low density makes sense: open maps contain many long straight corridors where JPS4 skips entire rows or columns in a single jump, while A* must still expand every cell.

At higher densities the gap narrows. With many obstacles, paths become more tortuous and corridors are shorter, so there is less symmetry for JPS4 to exploit. At $\rho = 0.40$ on the 100×100 grid, JPS4 still reduces expansions by 51% over Dijkstra and 39% over A*, but the advantage is smaller than at low density.

Figure 3 shows the expansion ratio relative to Dijkstra across both grid sizes, and Figure 4 isolates JPS4's advantage over A*.

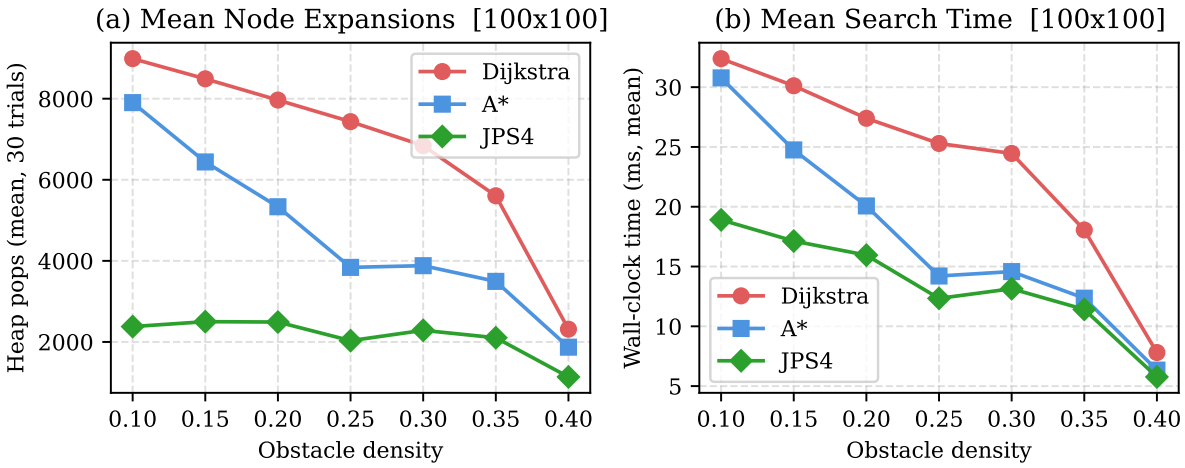

Figure 2: Same metrics on the 100×100 grid. The separation between JPS4 and the other two algorithms is more pronounced at this scale.

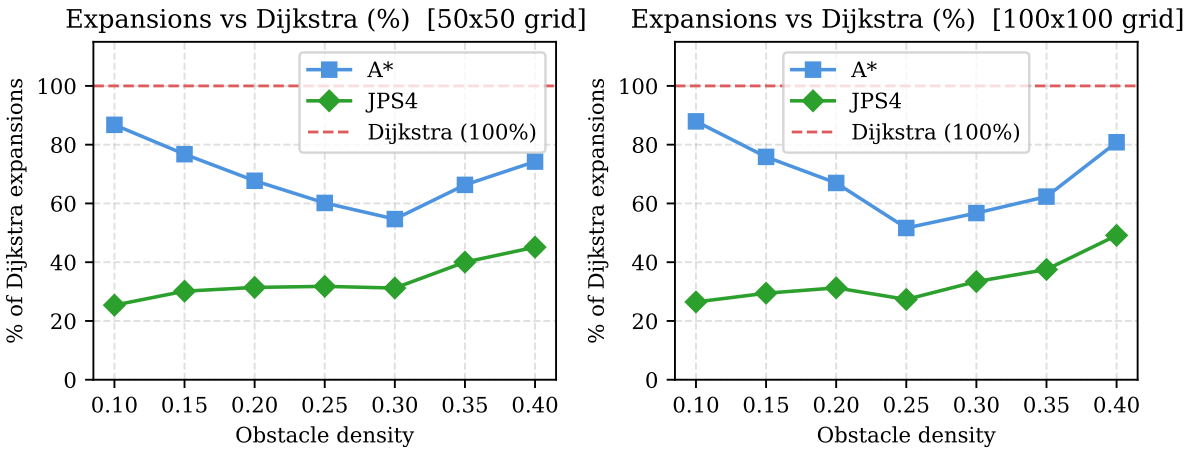

Figure 3: Node expansions of A* and JPS4 expressed as a percentage of Dijkstra's count. Values below 100% mean fewer expansions than Dijkstra. JPS4 is consistently the lowest.

5.2 Wall-Clock Time

Wall-clock time tracks the expansion reduction but with more noise. On the 50×50 grid, Dijkstra runs in roughly 6–8 ms at low density, A* in 4–6 ms, and JPS4 in 3–4 ms. The time gap is proportionally smaller than the expansion gap because JPS4's jump logic is more expensive per iteration than A*'s simple 4-neighbor lookup.

At $\rho = 0.35$ on the 50×50 grid, JPS4 and A* produce nearly identical runtimes (2.75 ms vs 2.70 ms) even though JPS4 expands 40% fewer nodes. This illustrates the overhead cost: each JPS4 step scans along a direction until it hits an obstacle or a forced neighbor, which is not free.

On the 100×100 grid the time advantage for JPS4 recovers. At $\rho = 0.10$, JPS4 takes 18.9 ms versus 30.8 ms for A* and 32.4 ms for Dijkstra. The reason is that on a larger grid, skipping a long corridor saves proportionally more total work, making the overhead cost worthwhile.

5.3 Path Correctness

Every trial confirmed that all three algorithms return the same path length. This is expected (all three are optimal on uniform-cost grids), but it serves as an ongoing regression check. The test suite formalizes

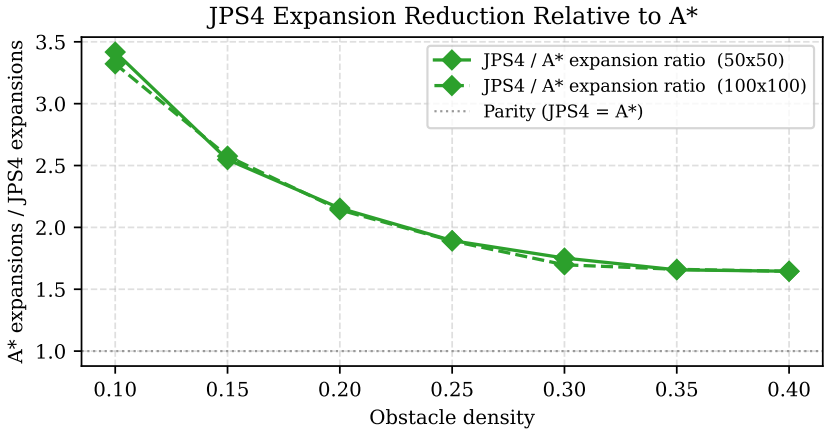

Figure 4: Ratio of A* expansions to JPS4 expansions. Values above 1.0 mean JPS4 expanded fewer nodes. The 100×100 grid shows consistently larger gains.

this: 500 random instances are checked each run, and the suite has found no path-length disagreements since the JPS4 implementation was finalized.

6 Discussion

6.1 When JPS4 Helps Most

The data consistently shows JPS4's advantage is largest on:

- **Larger grids.** The 100×100 grid sees bigger relative gains than 50×50 at every density. Longer corridors mean more nodes per jump.
- **Lower obstacle density.** Open maps have more straight-line symmetry, so more intermediate nodes can be pruned. At $\rho = 0.10$ on 100×100, JPS4 expands 3.4× fewer nodes than Dijkstra.

6.2 When JPS4 Helps Less

At high density ($\rho \geq 0.35$) the paths become twisty and jump opportunities shrink. JPS4 still wins on expansion count, but the per-step overhead narrows the wall-clock gap noticeably on smaller grids. In such conditions, standard A* is a simpler and nearly as fast choice.

This matches the observation in Baum (2025): JPS4 is not a universal replacement for A*; it is a specialization for maps with enough open space or repeated symmetry.

6.3 Dijkstra as a Baseline

Dijkstra's role here is mostly pedagogical. It confirms that the heuristic in A* actually helps (A* consistently beats Dijkstra on expansions), and it gives both A* and JPS4 a common reference point. In a real system you would not use Dijkstra for single-query grid pathfinding, but it is the clearest illustration of what the heuristic and the pruning are each contributing.

6.4 Limitations

- **Implementation language.** Everything runs in interpreted Python. Absolute times are not representative of C or C++ performance. Ratios and expansion counts are more meaningful.
- **Synthetic grids.** Random independent-cell obstacle placement produces a specific texture. Real maps (mazes, rooms, corridors) would likely show different density-performance relationships.

- **Single heuristic weight.** We used $w = 1$ for A* (exact admissible heuristic). Weighted A* ($w > 1$) trades optimality for speed and is not evaluated here.
- **No preprocessing.** JPS4 is an online method. Algorithms that allow offline preprocessing (such as HPA* or Contraction Hierarchies) are not compared.

7 Conclusion

We implemented and benchmarked Dijkstra, A*, and JPS4 on uniform-cost 4-connected grid maps across 420 problem instances per grid size. The results confirm that JPS4 reduces node expansions substantially relative to both Dijkstra and A* across all tested densities, with the largest gains on larger, more open maps. The reduction in wall-clock time is real but smaller, because JPS4's jump logic costs more per step than a plain 4-neighbor expansion.

For a practical recommendation: on 4-connected uniform-cost grids where maps are reasonably open and path queries are frequent, JPS4 is the right choice. On dense or heavily cluttered maps, or when simplicity matters, A* with Manhattan distance is hard to beat. Dijkstra belongs in the textbook.

The Snake demo makes the tradeoff concrete and visible: on a sparse board you can watch JPS4 finish its search while A* is still expanding nodes, and on a crowded board the three algorithms converge to nearly the same behavior.

References

References

[1] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959. https://doi.org/10.1007/BF01386390

[2] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. https://doi.org/10.1109/TSSC.1968.300136

[3] D. Harabor and A. Grastien. Online graph pruning for pathfinding on grid maps. In *Proceedings of the 25th AAAI Conference on Artificial Intelligence*, pages 1114–1119, 2011. https://harabor.net/data/papers/harabor-grastien-aaai11.pdf

[4] M. Baum. Jump point search pathfinding in 4-connected grids. *arXiv preprint arXiv:2501.14816*, 2025. https://arxiv.org/abs/2501.14816